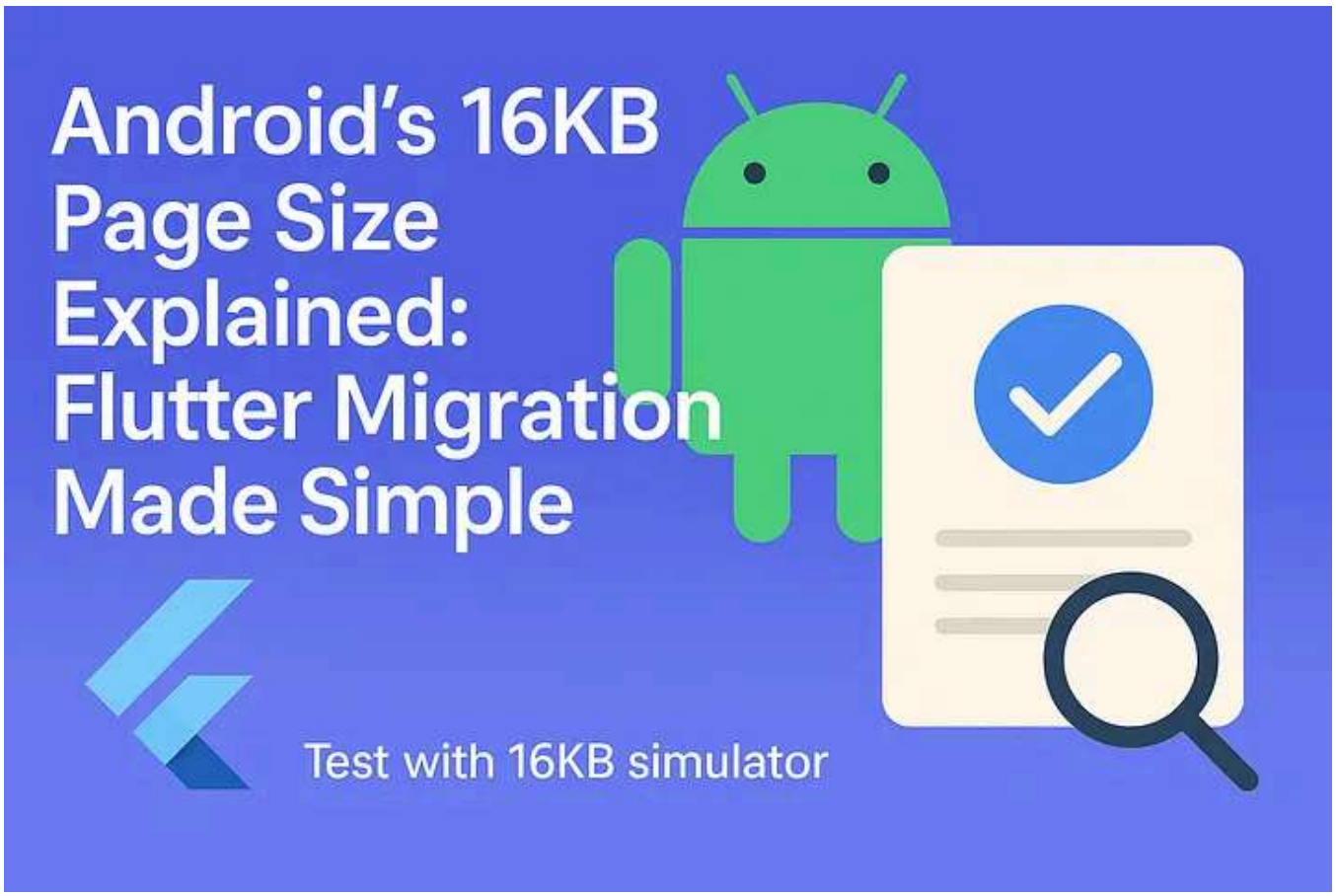
Easy Flutter · Follow publication

Android's 16KB Page Size Explained: Flutter Migration Made Simple

5 min read · Sep 10, 2025


Ravi Pai Follow

Listen Share



Google announced that starting **November 1, 2025, all new app submissions and updates to Google Play must support the new 16KB memory page size** on **Android 15+ (API level 35) 64-bit devices.**

This means if your Flutter app (or its dependencies) isn't 16KB-ready, you risk:

- Google Play **rejecting your build**.
- Crashes or failed installs on newer devices.
- A sudden all-nighter you didn't plan for.

So let's break this down, **what 16KB is**, **why it matters**, and most importantly, **how to migrate your Flutter app safely**.

What Even Is Page Size?.

In operating systems, **memory is divided into chunks called pages**.

- **Old standard:** 4KB pages → small chunks, more overhead.
- **New standard:** 16KB pages → bigger chunks, fewer lookups, faster access.

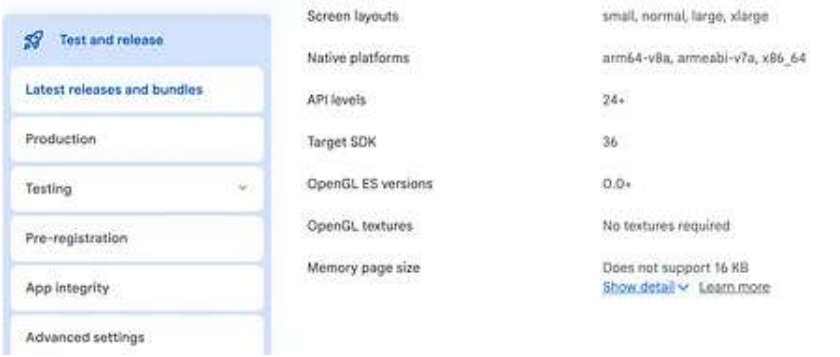
Why Google cares:

- App launches: **up to 30% faster**.
- Battery use: **~5% lower** during startup.
- Camera startup: **~5–7% faster**.
- System boot: **~1s faster**.

For users, this means speed and efficiency. For devs… it means work.

Check if Your App Requires Migration.

If your app is already live on the Play Store (of course it is — otherwise why are you even stressing about this? 😉), checking is super easy. Just head over to the **Play Console**, open your **latest release**, and in the **Bundle section** click the little arrow next to your app bundle. Scroll down to the **App details** and you'll spot a **Memory page size** field. That's where Google politely tells you whether your app is a 16KB-friendly citizen or still stuck in the 4KB Stone Age.



App detail page

The second way is for those of us who trust Android Studio more than our morning coffee ☕. Just make sure you're on the **latest version** (because of course Google loves hiding new features behind updates). Then fire up the **App Analyzer**:

1. Go to **Build > Analyze APK...**
2. Open your release **APK/AAB**.
3. Navigate to the **lib/** folder to see all the `.so` libraries hiding inside.



If you find suspicious `.so` files lurking around, congrats — you've just discovered the freeloaders in your app bundle that might break 16KB support.

Upgrade Your Toolchain

Now that you've survived the detective work and figured out which parts of your app are causing trouble, it's time to roll up your sleeves and make some changes — yes, the "fun" part where you actually fix things. 😅

The Easiest Way to Survive the 16KB Page Size Migration

Think of this like sending your app to a spa retreat — fresh tools, fewer complaints, and smoother performance. Instead of chasing down obscure `.so` file issues, just **upgrade everything to versions that fully support 16KB.**

Get Ravi Pai's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Here's what you need:

- **Android Gradle Plugin (AGP):** `8.5.1+`
- **Gradle Wrapper:** `8.5+`
- **NDK:** `r28+` (older ones can't align binaries for 16KB correctly)
- **Flutter:** `3.32+`

Open in app ↗ Sign up Sign in

Medium Search

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.13-all.zip
```

Step 2: Update Android Gradle Plugin in `build.gradle`

```
plugins {

    id "com.android.application" version '8.12.2' apply false
    id "org.jetbrains.kotlin.android" version "2.2.10" apply false
  }
```

Step 3: Point to the New NDK

```
android {
ndkVersion "29.0.13113456 rc1"
}
```

After some digging and a fair amount of hair-pulling 😅, I discovered this neat command that helped me pinpoint the plugins responsible for stubborn `.so` files:

```
find . -name "libnative-lib.so"
```



After doing all the dance 💃🕺 with upgrades and configuration tweaks, it's finally time to **verify** that your app is ready for the 16KB world.

Step 1: Clean Build Process.

```
# Clean previous builds
./gradlew clean

# Rebuild with new configuration
./gradlew assembleRelease
```

Step 2: Verification

You can find the `check_elf_alignment.sh` script here — special thanks to **@NitinPrakash9911** for sharing it.

Place it in the **root of your project**, then make it executable:

```
chmod +x check_elf_alignment.sh
```

```
./check_elf_alignment.sh app/build/outputs/apk/release/app-release.apk
```

> *Note: Paste your app path after the ./check_elf_alignment.sh*

If everything is configured correctly, you should see this output:



Migrating your Flutter app for the 16KB page size might sound scary at first — Play Console warnings, stubborn `.so` files, and endless “why is this crashing?” moments — but armed with the right tools and a bit of detective work, it’s totally doable.